LAPORAN PROJECT 1

ANALISIS DAN IMPLEMENTASI ALGORITMA KRIPTOGRAFI SIMETRIS ADVANCED ENCRYPTION STANDARD (AES)

MATA KULIAH KRIPTOGRAFI

Disusun untuk memenuhi tugas Project 1 Mata Kuliah Kriptografi



Dosen Pengampu:

Ir. Nizirwan Anwar, MT, IPM, ASEAN.Eng., APEC Eng.

Disusun Oleh:

20240801102 - Ilham Firmansyah

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

ABSTRAK

Perkembangan teknologi informasi menyebabkan pertukaran dan penyimpanan data digital semakin banyak dilakukan melalui sistem komputer dan jaringan. Kondisi tersebut menjadikan keamanan informasi sebagai salah satu aspek penting yang perlu diperhatikan. Salah satu pendekatan yang digunakan untuk menjaga kerahasiaan data adalah kriptografi, khususnya algoritma kriptografi kunci simetris. Advanced Encryption Standard (AES) merupakan algoritma enkripsi simetris berbasis blok yang digunakan untuk melindungi informasi digital melalui proses transformasi data menggunakan kunci kriptografi.

Project ini bertujuan untuk mempelajari konsep dasar AES, menganalisis tahapan transformasi dalam proses enkripsi dan dekripsi, serta menerapkan algoritma AES menggunakan bahasa pemrograman Python. Pembahasan mencakup karakteristik AES-128, AES-192, dan AES-256, struktur algoritma, proses Key Expansion, AddRoundKey, SubBytes, ShiftRows, MixColumns, serta transformasi inversnya. Selain analisis algoritma, dilakukan pula implementasi AES-256 dalam mode Galois/Counter Mode (GCM) menggunakan pustaka kriptografi Python.

Pengujian dilakukan menggunakan teks uji berupa “ILHAM FIRMANSYAH - UTS KRIPTOGRAFI”. Program menghasilkan ciphertext, nonce, dan authentication tag, kemudian melakukan proses dekripsi untuk memperoleh kembali plaintext. Pengujian tambahan dilakukan dengan mengubah ciphertext, authentication tag, kunci, dan nonce untuk memastikan bahwa perubahan data dapat dideteksi oleh mekanisme autentikasi AES-GCM. Berdasarkan pengujian yang dilakukan, lima skenario pengujian otomatis berhasil dilewati.

Hasil project menunjukkan bahwa AES dapat digunakan sebagai algoritma untuk menjaga kerahasiaan data, sedangkan penggunaan mode GCM memberikan tambahan kemampuan verifikasi integritas dan autentikasi data. Implementasi melalui pustaka kriptografi juga membantu mengurangi risiko kesalahan yang dapat terjadi apabila algoritma kriptografi dibuat sendiri tanpa proses validasi yang memadai.

Kata kunci: Kriptografi, Advanced Encryption Standard, AES-256, AES-GCM, Enkripsi, Dekripsi, Python.

DAFTAR ISI

ABSTRAK ..... **ii**

DAFTAR ISI ..... **iii**

DAFTAR TABEL ..... **vii**

DAFTAR GAMBAR ..... **viii**

BAB I PENDAHULUAN **9**

1.1 Latar Belakang 9

1.2 Rumusan Masalah 10

1.3 Tujuan Project 11

1.4 Manfaat Project 11

1. Manfaat Akademis 11

2. Manfaat Praktis 11

3. Manfaat Keamanan Informasi 11

1.5 Batasan Masalah 12

BAB II LANDASAN TEORI **13**

2.1 Pengertian Kriptografi 13

2.2 Kriptografi Simetris dan Asimetris 14

2.2.1 Kriptografi Simetris 14

2.2.2 Kriptografi Asimetris 14

2.2.3 Perbedaan Kriptografi Simetris dan Asimetris 15

2.3 Advanced Encryption Standard (AES) 15

2.4 Struktur Substitution-Permutation Network (SPN) 16

2.5 Ukuran Blok dan Ukuran Kunci AES 16

2.5.1 AES-128 17

2.5.2 AES-192 17

2.5.3 AES-256 17

2.6 Representasi Data dalam AES 17

2.7 Tahapan Utama Algoritma AES 18

2.7.1 Key Expansion 18

2.7.2 AddRoundKey 19

2.7.3 SubBytes 19
2.7.4 ShiftRows 20
2.7.5 MixColumns 20
2.8 Proses Enkripsi AES 21
2.8.1 Initial Round 21
2.8.2 Main Rounds 21
2.8.3 Final Round 22
2.9 Proses Dekripsi AES 22
2.9.1 InvShiftRows 22
2.9.2 InvSubBytes 22
2.9.3 AddRoundKey 22
2.9.4 InvMixColumns 23
2.10 Perbedaan Putaran Awal, Tengah, dan Akhir 23
2.11 Mode Operasi AES 23
2.11.1 Electronic Codebook (ECB) 24
2.11.2 Cipher Block Chaining (CBC) 24
2.11.3 Counter Mode (CTR) 24
2.11.4 Galois/Counter Mode (GCM) 24
BAB III ANALISIS ALGORITMA AES 26
3.1 Gambaran Umum Alur Algoritma AES 26
3.2 Diagram Alur Dekripsi AES 28
3.3 Analisis Key Expansion 29
3.4 Analisis SubBytes 29
3.5 Analisis ShiftRows 30
3.6 Analisis MixColumns 31
3.7 Analisis AddRoundKey 31
3.8 Analisis Perbedaan AES-128, AES-192, dan AES-256 32
3.9 Analisis Hubungan Enkripsi dan Dekripsi 32
BAB IV METODE DAN STUDI KASUS 33
4.1 Metode Pengerjaan 33
4.2 Lingkungan Pengembangan 33

4.3 Studi Kasus 34
4.3.1 Data Plaintext 34
4.3.2 Pembangkitan Kunci 35
4.3.3 Pembangkitan Nonce 35
4.3.4 Proses Enkripsi 35
4.3.5 Proses Dekripsi 35
4.4 Representasi Hasil Studi Kasus 36
4.5 Verifikasi Hasil Dekripsi 37
BAB V IMPLEMENTASI DAN PENGUJIAN PROGRAM 38
5.1 Perancangan Program 38
5.2 Struktur Folder Project 38
5.3 Persiapan Lingkungan Python 39
1. Memeriksa Python 39
2. Membuat Virtual Environment 39
3. Mengaktifkan Virtual Environment 39
4. Memasang Library 39
5. Menyimpan Dependensi 39
5.4 Implementasi Fungsi Pembangkitan Kunci 39
5.5 Implementasi Fungsi Enkripsi 39
5.6 Implementasi Fungsi Dekripsi 41
5.7 Implementasi Alur Program Utama 42
5.8 Implementasi Unit Testing 43
5.8.1 Pengujian Enkripsi dan Dekripsi 43
5.8.2 Pengujian Perubahan Ciphertext 43
5.8.3 Pengujian Perubahan Authentication Tag 43
5.8.4 Pengujian Penggunaan Kunci yang Salah 43
5.8.5 Pengujian Penggunaan Nonce yang Salah 44
5.9 Hasil Pengujian 44
5.10 Analisis Hasil Pengujian 45
BAB VI EVALUASI KEAMANAN DAN PEMBAHASAN 46
6.1 Kerahasiaan Data (Confidentiality) 46

6.2 Integritas Data (Integrity) 46
6.3 Autentikasi Data 46
6.4 Risiko Penggunaan ECB 47
6.5 Pentingnya Penggunaan Nonce yang Unik 47
6.6 Pengelolaan Kunci 47
6.7 Kelebihan Implementasi AES-256-GCM 48
1. Menggunakan Algoritma Standar 48
2. Menggunakan Kunci 256 Bit 48
3. Mendukung Kerahasiaan dan Autentikasi 48
4. Menggunakan Pustaka Kriptografi 48
5. Memiliki Pengujian Otomatis 48
6.8 Keterbatasan Implementasi 49
6.9 Hubungan Analisis AES dan Implementasi AES-GCM 49
BAB VII KESIMPULAN DAN SARAN 50
7.1 Kesimpulan 50
7.2 Saran 51
DAFTAR PUSTAKA 52
LAMPIRAN 53
Lampiran A. Dokumentasi Struktur Repository 53
Lampiran B. Dokumentasi Proses Enkripsi dan Dekripsi 54
Lampiran C. Dokumentasi Unit Testing 55
Lampiran D. Source Code Program 56
Lampiran E. Petunjuk Menjalankan Program 57

DAFTAR TABEL

Tabel 1. Perbedaan Kriptografi Simetris dan Asimetris 15

Tabel 2. Ukuran Blok dan Ukuran Kunci AES..... 16

Tabel 3. Representasi Data dalam AES 17

Tabel 4. Perbedaan Putaran Awal, Tengah, dan Akhir..... 23

Tabel 5. Analisis Key Expansion..... 29

Tabel 6. Analisis ShiftRows..... 30

Tabel 7. Analisis Perbedaan AES-128, AES-192, dan AES-256 32

Tabel 8. Lingkungan Pengembangan..... 33

Tabel 9. Informasi Data Uji 36

Tabel 10. Contoh Hasil Enkripsi..... 36

Tabel 11. Hasil Pengujian Unit Test 44

DAFTAR GAMBAR

Gambar 1. ShiftRows 20

Gambar 2. MixColumns 21

Gambar 3. Diagram Alur Enkripsi AES 27

Gambar 4. Diagram Alur Dekripsi AES 28

Gambar 5. Ilustrasi Transformasi SubBytes 30

Gambar 6. Ilustrasi Transformasi ShiftRows 30

Gambar 7. Ilustrasi Transformasi MixColumns 31

Gambar 8. Tampilan Program AES-256-GCM 42

Gambar 9. Hasil Pengujian Otomatis 45

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah membawa perubahan besar dalam aktivitas manusia, terutama dalam proses pengolahan, penyimpanan, dan pertukaran informasi. Data digital saat ini digunakan dalam berbagai bidang, seperti pendidikan, pemerintahan, bisnis, komunikasi, dan layanan digital. Kemudahan dalam mengakses dan mengirimkan informasi juga diikuti dengan meningkatnya kebutuhan terhadap sistem keamanan yang dapat melindungi data dari akses maupun perubahan yang tidak diinginkan.

Informasi yang disimpan atau dikirimkan melalui sistem komputer tidak selalu bersifat publik. Beberapa informasi memiliki nilai penting dan hanya boleh diketahui oleh pihak tertentu. Apabila data tersebut dapat dibaca oleh pihak yang tidak berwenang, maka kerahasiaan informasi dapat terganggu. Selain itu, perubahan data selama proses penyimpanan maupun pengiriman dapat menyebabkan informasi yang diterima tidak lagi sesuai dengan data aslinya.

Salah satu bidang ilmu yang digunakan untuk mengatasi permasalahan tersebut adalah kriptografi. Kriptografi merupakan ilmu yang mempelajari teknik untuk mengamankan informasi melalui proses matematis sehingga data dapat diubah menjadi bentuk yang tidak mudah dipahami oleh pihak yang tidak memiliki kewenangan. Dalam penerapannya, kriptografi menggunakan algoritma dan kunci tertentu untuk melakukan proses enkripsi dan dekripsi [1].

Salah satu algoritma kriptografi yang banyak digunakan adalah Advanced Encryption Standard atau AES. AES merupakan algoritma kriptografi kunci simetris berbasis blok yang menggunakan ukuran blok tetap sebesar 128 bit. AES memiliki tiga variasi panjang kunci, yaitu AES-128, AES-192, dan AES-256. Ketiga variasi tersebut memiliki jumlah putaran yang berbeda, yaitu masing-masing 10, 12, dan 14 putaran [2].

Dalam proses enkripsi, AES melakukan beberapa tahapan transformasi terhadap data yang direpresentasikan dalam bentuk matriks internal bernama State. Transformasi tersebut meliputi SubBytes, ShiftRows, MixColumns, dan AddRoundKey. Selain itu, AES menggunakan proses Key Expansion untuk menghasilkan serangkaian round key yang digunakan pada setiap putaran enkripsi.

Pemahaman mengenai AES tidak cukup hanya dengan mengetahui bahwa algoritma tersebut dapat mengubah plaintext menjadi ciphertext. Setiap tahapan transformasi

memiliki fungsi tersendiri dalam membentuk proses pengamanan data. Oleh karena itu, diperlukan analisis terhadap struktur algoritma dan tahapan enkripsi maupun dekripsi agar cara kerja AES dapat dipahami secara lebih menyeluruh.

Selain analisis teori, penerapan algoritma kriptografi juga perlu memperhatikan mode operasi yang digunakan. AES merupakan algoritma block cipher, sehingga penggunaannya pada data nyata memerlukan mode operasi yang sesuai. Salah satu mode yang direkomendasikan untuk memberikan kerahasiaan sekaligus autentikasi adalah Galois/Counter Mode (GCM). Mode ini menggabungkan proses enkripsi dengan mekanisme autentikasi untuk mendeteksi perubahan pada data [3].

Berdasarkan hal tersebut, project ini membahas analisis algoritma AES serta implementasinya menggunakan bahasa pemrograman Python. Implementasi dilakukan menggunakan pustaka cryptography dengan algoritma AES-256-GCM. Program dibuat untuk melakukan pembangkitan kunci, enkripsi, dekripsi, dan pengujian terhadap perubahan data. Dengan demikian, project ini diharapkan dapat memberikan pemahaman mengenai konsep AES sekaligus penerapannya dalam program sederhana.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam project ini adalah sebagai berikut:

1. Apa yang dimaksud dengan kriptografi kunci simetris dan bagaimana karakteristik algoritma AES?
2. Bagaimana struktur dan prinsip kerja algoritma AES dalam proses enkripsi dan dekripsi?
3. Apa perbedaan AES-128, AES-192, dan AES-256 berdasarkan panjang kunci dan jumlah putarannya?
4. Bagaimana fungsi Key Expansion, AddRoundKey, SubBytes, ShiftRows, dan MixColumns dalam proses AES?
5. Bagaimana penerapan AES-256-GCM menggunakan bahasa pemrograman Python?
6. Bagaimana hasil pengujian enkripsi, dekripsi, dan autentikasi data pada program yang dibuat?
7. Apa saja aspek keamanan dan keterbatasan yang perlu diperhatikan dalam penggunaan AES-GCM?

1.3 Tujuan Project

Tujuan dari pelaksanaan project ini adalah:

1. Memahami konsep dasar kriptografi, khususnya kriptografi kunci simetris.
2. Menjelaskan karakteristik dan struktur algoritma Advanced Encryption Standard.
3. Memahami perbedaan AES-128, AES-192, dan AES-256.
4. Menganalisis tahapan transformasi yang digunakan dalam proses enkripsi dan dekripsi AES.
5. Menerapkan algoritma AES-256-GCM menggunakan bahasa pemrograman Python.
6. Melakukan pengujian terhadap proses enkripsi dan dekripsi untuk memastikan kesesuaian hasil.
7. Menganalisis kemampuan AES-GCM dalam mendeteksi perubahan data melalui authentication tag.
8. Mengidentifikasi aspek keamanan dan keterbatasan penggunaan AES dalam implementasi sederhana.

1.4 Manfaat Project

Project ini diharapkan memberikan beberapa manfaat sebagai berikut.

1. Manfaat Akademis

Project ini dapat meningkatkan pemahaman mengenai konsep kriptografi simetris, khususnya algoritma AES. Pembahasan mengenai transformasi internal AES dapat membantu memahami hubungan antara teori matematika kriptografi dan penerapannya dalam sistem komputer.

2. Manfaat Praktis

Project ini memberikan pengalaman dalam menerapkan algoritma AES menggunakan bahasa pemrograman Python dan pustaka kriptografi yang telah tersedia. Selain itu, project ini memberikan pemahaman mengenai proses pengamanan data melalui enkripsi dan dekripsi.

3. Manfaat Keamanan Informasi

Project ini memberikan pemahaman mengenai pentingnya penggunaan mode operasi yang tepat, pengelolaan kunci, penggunaan nonce yang benar, serta verifikasi integritas data dalam penerapan kriptografi.

1.5 Batasan Masalah

Agar pembahasan lebih terarah, project ini memiliki beberapa batasan sebagai berikut:

1. Pembahasan difokuskan pada algoritma Advanced Encryption Standard (AES).
2. Variasi AES yang dianalisis meliputi AES-128, AES-192, dan AES-256.
3. Analisis teori mencakup struktur State, Key Expansion, AddRoundKey, SubBytes, ShiftRows, MixColumns, serta transformasi inversnya.
4. Implementasi program menggunakan bahasa pemrograman Python.
5. Penerapan praktikum menggunakan AES-256 dalam mode Galois/Counter Mode (GCM).
6. Pustaka yang digunakan adalah cryptography.
7. Data yang digunakan merupakan data uji dan bukan data rahasia atau data produksi.
8. Pengujian berfokus pada keberhasilan enkripsi dan dekripsi serta kemampuan mendeteksi perubahan ciphertext, authentication tag, kunci, dan nonce.
9. Project ini tidak melakukan pengujian penetrasi, analisis kriptografi tingkat lanjut, maupun pengukuran performa pada skala produksi.

BAB II
LANDASAN TEORI

2.1 Pengertian Kriptografi

Kriptografi merupakan bidang ilmu yang mempelajari teknik pengamanan informasi dengan menggunakan metode matematis dan algoritma tertentu. Tujuan utamanya adalah mengubah informasi asli menjadi bentuk yang tidak dapat dipahami secara langsung oleh pihak yang tidak memiliki hak akses.

Dalam kriptografi terdapat beberapa istilah penting yang digunakan untuk menjelaskan proses pengamanan informasi.

1. **Plaintext**, yaitu data asli yang akan diamankan atau dienkripsi.
2. **Ciphertext**, yaitu hasil perubahan plaintext setelah melalui proses enkripsi.
3. **Encryption**, yaitu proses mengubah plaintext menjadi ciphertext menggunakan algoritma dan kunci tertentu.
4. **Decryption**, yaitu proses mengembalikan ciphertext menjadi plaintext menggunakan algoritma dan kunci yang sesuai.
5. **Key**, yaitu nilai yang digunakan oleh algoritma kriptografi untuk mengendalikan proses enkripsi dan dekripsi.
6. **Nonce**, yaitu nilai yang digunakan satu kali dalam suatu proses kriptografi sesuai ketentuan algoritma dan mode yang digunakan.
7. **Authentication Tag**, yaitu nilai autentikasi yang digunakan untuk memeriksa apakah data yang diterima masih sesuai dengan data yang telah diautentikasi.

Secara umum, proses kriptografi dapat digambarkan melalui hubungan berikut:

$$C = E_K(P)$$

Keterangan:

- (P) = Plaintext.
- (C) = Ciphertext.
- (E) = Fungsi enkripsi.
- (K) = Kunci kriptografi.

Untuk mengembalikan ciphertext menjadi plaintext digunakan proses dekripsi:

$$P = D_K(C)$$

Keterangan:

- (D) = Fungsi dekripsi.
- (C) = Ciphertext.
- (K) = Kunci yang sesuai.
- (P) = Plaintext hasil dekripsi.

Dalam algoritma kunci simetris, kunci yang sama digunakan dalam proses enkripsi dan dekripsi. Oleh karena itu, pengelolaan kunci menjadi salah satu aspek penting dalam keamanan sistem kriptografi [1].

2.2 Kriptografi Simetris dan Asimetris

Berdasarkan penggunaan kuncinya, kriptografi dapat dibedakan menjadi dua kelompok utama, yaitu kriptografi simetris dan kriptografi asimetris.

2.2.1 Kriptografi Simetris

Kriptografi simetris merupakan metode kriptografi yang menggunakan satu kunci yang sama untuk proses enkripsi dan dekripsi. Pengirim dan penerima harus memiliki kunci yang sesuai agar data dapat diamankan dan dikembalikan ke bentuk aslinya.

Keunggulan kriptografi simetris adalah proses komputasinya relatif efisien sehingga sesuai untuk mengamankan data dalam jumlah besar. Namun, metode ini memiliki tantangan dalam pendistribusian dan penyimpanan kunci karena pihak yang berkomunikasi harus memiliki kunci yang sama secara aman.

AES merupakan salah satu contoh algoritma kriptografi simetris.

2.2.2 Kriptografi Asimetris

Kriptografi asimetris menggunakan pasangan kunci yang berbeda, yaitu public key dan private key. Public key dapat dibagikan kepada pihak lain, sedangkan private key harus dijaga oleh pemiliknya.

Pada sistem enkripsi asimetris, data dapat dienkripsi menggunakan public key penerima dan didekripsi menggunakan private key yang sesuai. Metode ini juga digunakan dalam berbagai mekanisme tanda tangan digital.

Contoh algoritma kriptografi asimetris adalah RSA dan Elliptic Curve Cryptography (ECC).

2.2.3 Perbedaan Kriptografi Simetris dan Asimetris

Tabel 1. Perbedaan Kriptografi Simetris dan Asimetris

Aspek	Simetris	Asimetris
Kunci	Satu kunci yang sama	Pasangan public key dan private key
Proses	Relatif efisien	Umumnya lebih kompleks
Pengelolaan kunci	Memerlukan pertukaran kunci secara aman	Menggunakan pasangan kunci
Penggunaan	Enkripsi data dalam jumlah besar	Pertukaran kunci dan tanda tangan digital
Contoh	AES, ChaCha20	RSA, ECC

Berdasarkan perbandingan tersebut, algoritma AES dipilih dalam project ini karena termasuk kriptografi simetris yang dapat digunakan untuk melakukan enkripsi dan dekripsi data secara efisien.

2.3 Advanced Encryption Standard (AES)

Advanced Encryption Standard atau AES merupakan standar algoritma kriptografi simetris yang ditetapkan oleh National Institute of Standards and Technology (NIST). AES berasal dari keluarga algoritma Rijndael yang dipilih sebagai standar enkripsi untuk melindungi informasi digital.

AES merupakan block cipher, yaitu algoritma yang memproses data dalam ukuran blok tertentu. Ukuran blok AES adalah 128 bit atau 16 byte. Algoritma ini mendukung tiga ukuran kunci, yaitu 128 bit, 192 bit, dan 256 bit [2].

AES memiliki beberapa karakteristik utama:

1. Menggunakan sistem kriptografi kunci simetris.
2. Memiliki ukuran blok tetap sebesar 128 bit.
3. Mendukung tiga ukuran kunci, yaitu 128, 192, dan 256 bit.
4. Menggunakan beberapa putaran transformasi untuk mengubah data.
5. Menggunakan proses Key Expansion untuk menghasilkan round key.
6. Memiliki tahapan enkripsi dan dekripsi yang saling berhubungan melalui transformasi invers.

AES dirancang untuk memberikan perlindungan terhadap kerahasiaan informasi. Kekuatan penggunaannya dipengaruhi oleh ukuran kunci, implementasi algoritma, pengelolaan kunci, dan mode operasi yang digunakan.

2.4 Struktur Substitution-Permutation Network (SPN)

AES memiliki struktur transformasi yang sering dijelaskan melalui pendekatan Substitution-Permutation Network atau SPN. Struktur ini menggabungkan proses substitusi dan permutasi untuk mengubah hubungan antara plaintext dan ciphertext.

Substitusi merupakan proses penggantian nilai berdasarkan aturan tertentu. Pada AES, proses ini dilakukan melalui transformasi SubBytes yang menggunakan tabel substitusi bernama S-Box.

Permutasi merupakan proses pengubahan posisi atau susunan elemen. Pada AES, perubahan posisi byte dilakukan melalui ShiftRows, sedangkan pencampuran nilai antarkolom dilakukan melalui MixColumns.

Kombinasi transformasi tersebut bertujuan menghasilkan perubahan yang menyebar ke berbagai bagian data sehingga hubungan antara plaintext, kunci, dan ciphertext menjadi tidak mudah dianalisis secara langsung.

AES tidak hanya melakukan satu kali substitusi atau permutasi, tetapi mengulangi beberapa transformasi dalam sejumlah putaran sesuai dengan panjang kunci yang digunakan [2].

2.5 Ukuran Blok dan Ukuran Kunci AES

AES memiliki ukuran blok tetap sebesar 128 bit. Artinya, setiap blok data yang diproses oleh inti algoritma AES memiliki ukuran 128 bit atau 16 byte.

Meskipun ukuran bloknya sama, AES memiliki tiga variasi panjang kunci. Perbedaan ukuran kunci tersebut menyebabkan jumlah putaran dan jumlah round key yang digunakan juga berbeda.

Tabel 2. Ukuran Blok dan Ukuran Kunci AES

Jenis AES	Panjang Kunci	Ukuran Kunci	Ukuran Blok	Jumlah Putaran
AES-128	128 bit	16 byte	128 bit	10
AES-192	192 bit	24 byte	128 bit	12
AES-256	256 bit	32 byte	128 bit	14

Perbedaan utama dari ketiga jenis AES terletak pada panjang kunci dan jumlah putaran yang digunakan. Ukuran blok tetap 128 bit pada seluruh variasi tersebut [2].

2.5.1 AES-128

AES-128 menggunakan kunci dengan panjang 128 bit atau 16 byte. Algoritma ini menjalankan 10 putaran transformasi setelah proses AddRoundKey awal.

2.5.2 AES-192

AES-192 menggunakan kunci dengan panjang 192 bit atau 24 byte. Algoritma ini menjalankan 12 putaran transformasi.

2.5.3 AES-256

AES-256 menggunakan kunci dengan panjang 256 bit atau 32 byte. Algoritma ini menjalankan 14 putaran transformasi.

Dalam implementasi praktikum project ini, variasi yang digunakan adalah AES-256 karena pustaka cryptography menyediakan dukungan untuk AES dengan ukuran kunci tersebut.

2.6 Representasi Data dalam AES

Sebelum diproses oleh AES, data harus direpresentasikan dalam bentuk byte. Pada data berbentuk teks, karakter terlebih dahulu dikonversi menggunakan encoding tertentu, seperti UTF-8.

Sebagai contoh, teks:

AES

Jika dikonversi menggunakan ASCII/UTF-8, masing-masing karakter memiliki nilai byte sebagai berikut:

Tabel 3. Representasi Data dalam AES

Karakter	Desimal	Heksadesimal	Biner
A	65	41	01000001
E	69	45	01000101
S	83	53	01010011

Setiap byte memiliki panjang 8 bit. Pada AES, 16 byte data dalam satu blok disusun menjadi matriks berukuran 4 × 4 byte yang disebut State.

Secara umum, State memiliki bentuk:

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Setiap elemen matriks State merupakan satu byte. Dengan demikian, total ukuran State adalah 16 byte atau 128 bit.

Representasi State menjadi dasar bagi transformasi yang dilakukan dalam setiap putaran AES [2].

2.7 Tahapan Utama Algoritma AES

Algoritma AES menggunakan beberapa transformasi utama untuk mengubah data. Transformasi tersebut memiliki fungsi yang berbeda dan saling melengkapi.

2.7.1 Key Expansion

Key Expansion merupakan proses pembentukan sejumlah round key dari kunci utama atau cipher key.

Kunci utama yang diberikan kepada algoritma tidak langsung digunakan secara berulang tanpa perubahan. AES terlebih dahulu memproses kunci tersebut melalui Key Expansion untuk menghasilkan kumpulan round key.

Round key digunakan dalam proses AddRoundKey pada putaran yang berbeda.

Proses Key Expansion melibatkan beberapa operasi, antara lain:

1. **RotWord**, yaitu menggeser susunan byte dalam satu word.
2. **SubWord**, yaitu mengganti setiap byte menggunakan S-Box.
3. **Rcon**, yaitu konstanta ronde yang digunakan dalam proses pembentukan word tertentu.
4. **XOR**, yaitu operasi Exclusive OR untuk menggabungkan nilai word.

Jumlah word yang dihasilkan disesuaikan dengan jenis AES yang digunakan.

AES-128 menggunakan 4 word untuk kunci utama, AES-192 menggunakan 6 word, sedangkan AES-256 menggunakan 8 word.

Hasil Key Expansion digunakan sebagai round key pada tahap AddRoundKey [2].

2.7.2 AddRoundKey

AddRoundKey merupakan proses menggabungkan setiap byte pada State dengan byte yang berasal dari round key menggunakan operasi XOR.

Operasi XOR menghasilkan nilai berdasarkan perbandingan dua bit:

Bit A	Bit B	Hasil XOR
0	0	0
0	1	1
1	0	1
1	1	0

Secara matematis, operasi AddRoundKey dapat dituliskan sebagai:

$$State' = State \oplus RoundKey$$

Operasi ini menghubungkan data yang sedang diproses dengan kunci kriptografi. Tanpa round key yang sesuai, proses dekripsi tidak dapat mengembalikan data ke bentuk aslinya.

AddRoundKey dilakukan pada awal proses dan pada setiap putaran sesuai struktur AES.

2.7.3 SubBytes

SubBytes merupakan proses substitusi setiap byte pada State menggunakan tabel substitusi yang disebut S-Box.

S-Box berisi pemetaan nilai byte dari 0 sampai 255 ke nilai byte lainnya. Proses ini memberikan perubahan nonlinier pada data.

Tujuan utama SubBytes adalah memperkuat sifat confusion, yaitu membuat hubungan antara plaintext, kunci, dan ciphertext menjadi lebih sulit dipahami.

Pada proses dekripsi digunakan transformasi kebalikan yang disebut InvSubBytes. Transformasi tersebut menggunakan tabel inverse S-Box untuk mengembalikan nilai byte.

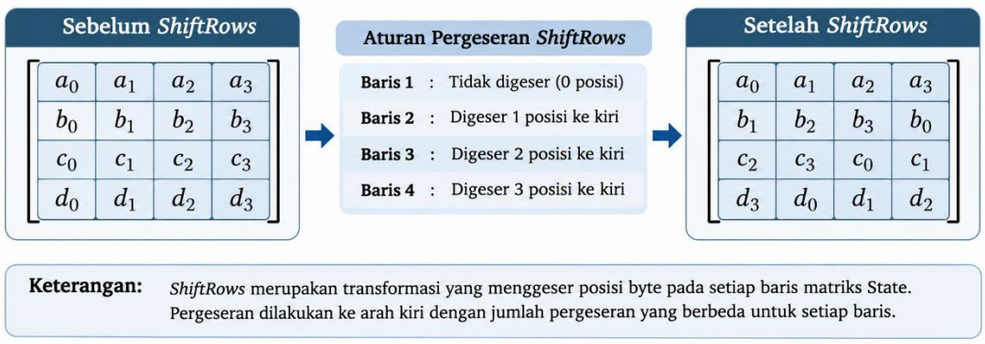
2.7.4 ShiftRows

ShiftRows merupakan proses pergeseran posisi byte pada setiap baris matriks State.

Aturan pergeserannya adalah:

1. Baris pertama tidak mengalami pergeseran.
2. Baris kedua digeser satu posisi ke kiri.
3. Baris ketiga digeser dua posisi ke kiri.
4. Baris keempat digeser tiga posisi ke kiri.

Contoh susunan sebelum ShiftRows:



Gambar 1. ShiftRows

Pergeseran tersebut menyebabkan byte berpindah posisi sehingga nilai yang sebelumnya berada dalam satu kolom dapat tersebar ke kolom lainnya.

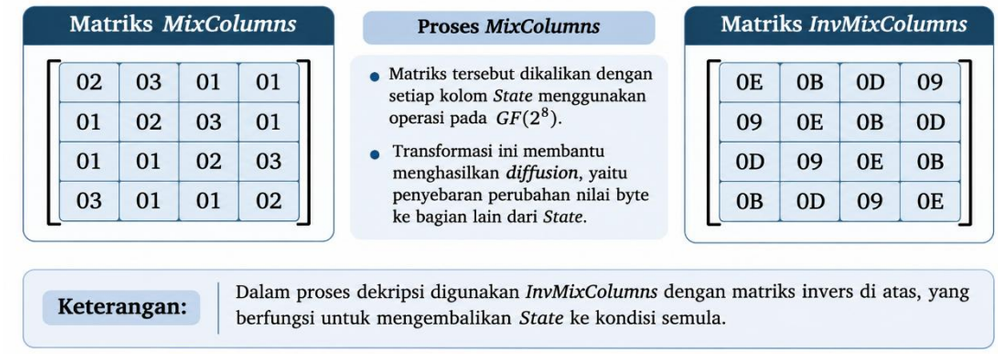
Pada proses dekripsi digunakan InvShiftRows, yaitu pergeseran ke arah sebaliknya.

2.7.5 MixColumns

MixColumns merupakan proses pencampuran nilai byte dalam setiap kolom State menggunakan operasi matematika pada medan hingga (GF(2^8)).

Pada tahap ini, setiap kolom diproses menggunakan perkalian matriks tetap. Tujuannya adalah menyebarkan pengaruh setiap byte ke beberapa byte lain dalam kolom yang sama.

Matriks transformasi MixColumns adalah:



Gambar 2. MixColumns

MixColumns digunakan pada putaran tengah, tetapi tidak digunakan pada putaran terakhir AES [2].

2.8 Proses Enkripsi AES

Proses enkripsi AES dilakukan melalui serangkaian transformasi terhadap State. Secara umum, tahapan enkripsi terdiri atas proses awal, putaran utama, dan putaran akhir.

2.8.1 Initial Round

Tahap awal dilakukan sebelum putaran utama. Pada tahap ini, State yang berisi data plaintext digabungkan dengan round key pertama melalui operasi AddRoundKey.

Proses tersebut memastikan bahwa kunci kriptografi mulai memengaruhi data sebelum transformasi putaran berikutnya dilakukan.

2.8.2 Main Rounds

Setelah proses awal, AES menjalankan putaran utama sesuai jumlah putaran yang ditentukan oleh jenis kunci.

Setiap putaran utama terdiri atas:

1. SubBytes.
2. ShiftRows.
3. MixColumns.
4. AddRoundKey.

Keempat transformasi tersebut dilakukan secara berurutan untuk menghasilkan perubahan terhadap State.

2.8.3 Final Round

Putaran terakhir memiliki susunan yang berbeda dengan putaran utama. Pada tahap ini, AES menjalankan:

1. SubBytes.
2. ShiftRows.
3. AddRoundKey.

MixColumns tidak dilakukan pada putaran terakhir.

Perbedaan tersebut merupakan bagian dari struktur standar AES [2].

2.9 Proses Dekripsi AES

Dekripsi merupakan proses mengembalikan ciphertext menjadi plaintext menggunakan kunci yang sesuai.

Dalam AES, proses dekripsi menggunakan transformasi invers dari proses enkripsi.

Tahapan utama dekripsi meliputi:

1. InvShiftRows.
2. InvSubBytes.
3. AddRoundKey.
4. InvMixColumns.

Urutan transformasi disesuaikan dengan struktur dekripsi AES dan penggunaan round key dalam urutan yang sesuai.

2.9.1 InvShiftRows

InvShiftRows merupakan transformasi kebalikan dari ShiftRows. Pada tahap ini, posisi byte pada setiap baris digeser ke arah kanan sesuai jumlah pergeseran yang ditentukan.

2.9.2 InvSubBytes

InvSubBytes merupakan transformasi kebalikan dari SubBytes. Setiap byte dipetakan menggunakan inverse S-Box untuk mengembalikan nilai sebelum proses substitusi.

2.9.3 AddRoundKey

AddRoundKey tetap menggunakan operasi XOR antara State dan round key yang sesuai. Karena XOR memiliki sifat invers terhadap dirinya sendiri, operasi yang sama dapat digunakan dalam proses enkripsi maupun dekripsi.

2.9.4 InvMixColumns

InvMixColumns merupakan transformasi kebalikan dari MixColumns. Proses ini menggunakan matriks invers untuk mengembalikan pencampuran byte yang dilakukan pada proses enkripsi.

Melalui penggunaan transformasi invers dan round key yang sesuai, ciphertext dapat dikembalikan menjadi plaintext [2].

2.10 Perbedaan Putaran Awal, Tengah, dan Akhir

Dalam algoritma AES, setiap jenis putaran memiliki susunan transformasi yang berbeda.

Tabel 4. Perbedaan Putaran Awal, Tengah, dan Akhir

Tahapan	Transformasi
Putaran awal	AddRoundKey
Putaran tengah	SubBytes, ShiftRows, MixColumns, AddRoundKey
Putaran akhir	SubBytes, ShiftRows, AddRoundKey

Putaran awal bertujuan menggabungkan data dengan round key pertama. Putaran tengah melakukan substitusi, pergeseran, pencampuran kolom, dan penggabungan kunci. Sementara itu, putaran akhir tidak menggunakan MixColumns.

Perbedaan ini berlaku pada struktur standar AES untuk ketiga variasi panjang kunci. Jumlah putaran utama yang dijalankan tetap bergantung pada jenis AES yang digunakan.

2.11 Mode Operasi AES

AES merupakan algoritma block cipher yang memproses data dalam blok berukuran 128 bit. Untuk menggunakannya dalam berbagai kebutuhan, diperlukan mode operasi yang menentukan bagaimana proses enkripsi diterapkan pada data.

Beberapa mode operasi yang dikenal antara lain:

1. Electronic Codebook (ECB).
2. Cipher Block Chaining (CBC).
3. Counter Mode (CTR).
4. Galois/Counter Mode (GCM).

Setiap mode memiliki karakteristik dan kebutuhan penggunaan yang berbeda. Pemilihan mode operasi harus mempertimbangkan kebutuhan kerahasiaan, integritas, dan autentikasi data.

2.11.1 Electronic Codebook (ECB)

ECB memproses setiap blok data secara terpisah menggunakan kunci yang sama. Kelemahan penting dari ECB adalah blok plaintext yang sama akan menghasilkan blok ciphertext yang sama apabila menggunakan kunci yang sama.

Akibatnya, pola tertentu dari data asli dapat terlihat pada ciphertext. Oleh karena itu, ECB tidak disarankan untuk mengamankan data yang memiliki pola berulang atau struktur tertentu [3].

2.11.2 Cipher Block Chaining (CBC)

CBC menggabungkan blok plaintext dengan hasil ciphertext dari blok sebelumnya sebelum proses enkripsi dilakukan. Mode ini menggunakan Initialization Vector (IV) pada blok pertama.

CBC dapat membantu mengurangi kemunculan pola yang sama seperti pada ECB, tetapi tetap memerlukan mekanisme autentikasi tambahan agar perubahan ciphertext dapat dideteksi.

2.11.3 Counter Mode (CTR)

CTR menggunakan nilai counter yang diproses bersama kunci untuk menghasilkan keystream. Hasil keystream kemudian digabungkan dengan plaintext menggunakan XOR.

CTR memungkinkan proses enkripsi dan dekripsi dilakukan secara efisien. Namun, penggunaan counter atau nonce harus dikelola dengan benar agar tidak terjadi penggunaan ulang yang dapat membahayakan keamanan.

2.11.4 Galois/Counter Mode (GCM)

GCM merupakan mode operasi yang menggabungkan enkripsi berbasis counter dengan mekanisme autentikasi. Mode ini dikenal sebagai authenticated encryption karena dapat memberikan kerahasiaan dan memeriksa integritas serta autentikasi data.

GCM menghasilkan ciphertext dan authentication tag. Saat proses dekripsi, tag diperiksa untuk memastikan data dan parameter autentikasi masih sesuai.

Jika terjadi perubahan pada ciphertext, tag, kunci, atau nonce, proses autentikasi dapat gagal [3].

Dalam project ini, AES-GCM dipilih karena sesuai dengan kebutuhan praktikum yang tidak hanya menunjukkan proses enkripsi dan dekripsi, tetapi juga pengujian terhadap perubahan data.

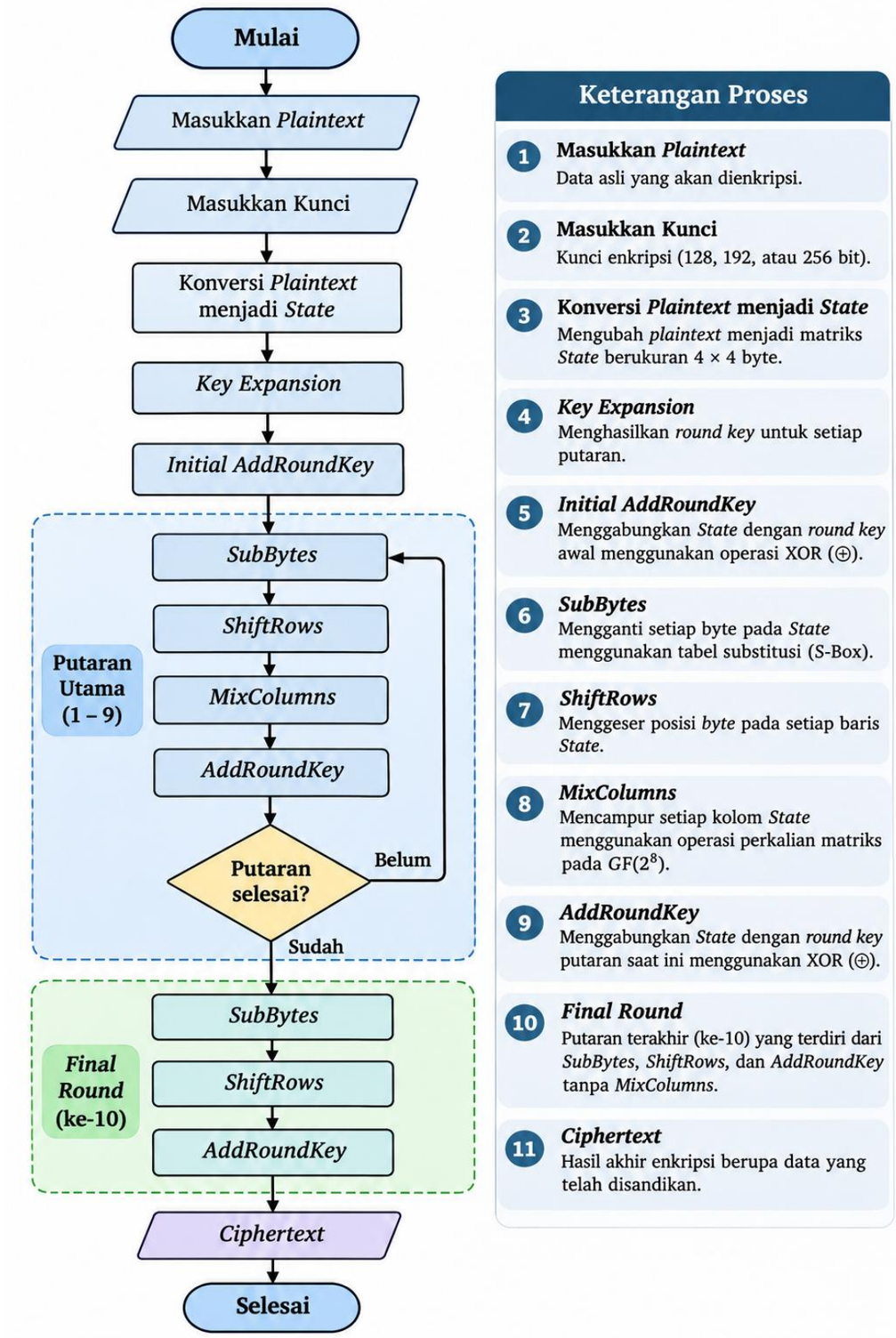
BAB III
ANALISIS ALGORITMA AES

3.1 Gambaran Umum Alur Algoritma AES

Secara umum, algoritma AES menerima plaintext dan kunci sebagai masukan. Plaintext diproses menjadi State, sedangkan kunci utama diproses melalui Key Expansion untuk menghasilkan round key.

Selanjutnya, State menjalani serangkaian transformasi hingga menghasilkan ciphertext.

Alur proses enkripsi AES dapat digambarkan sebagai berikut.



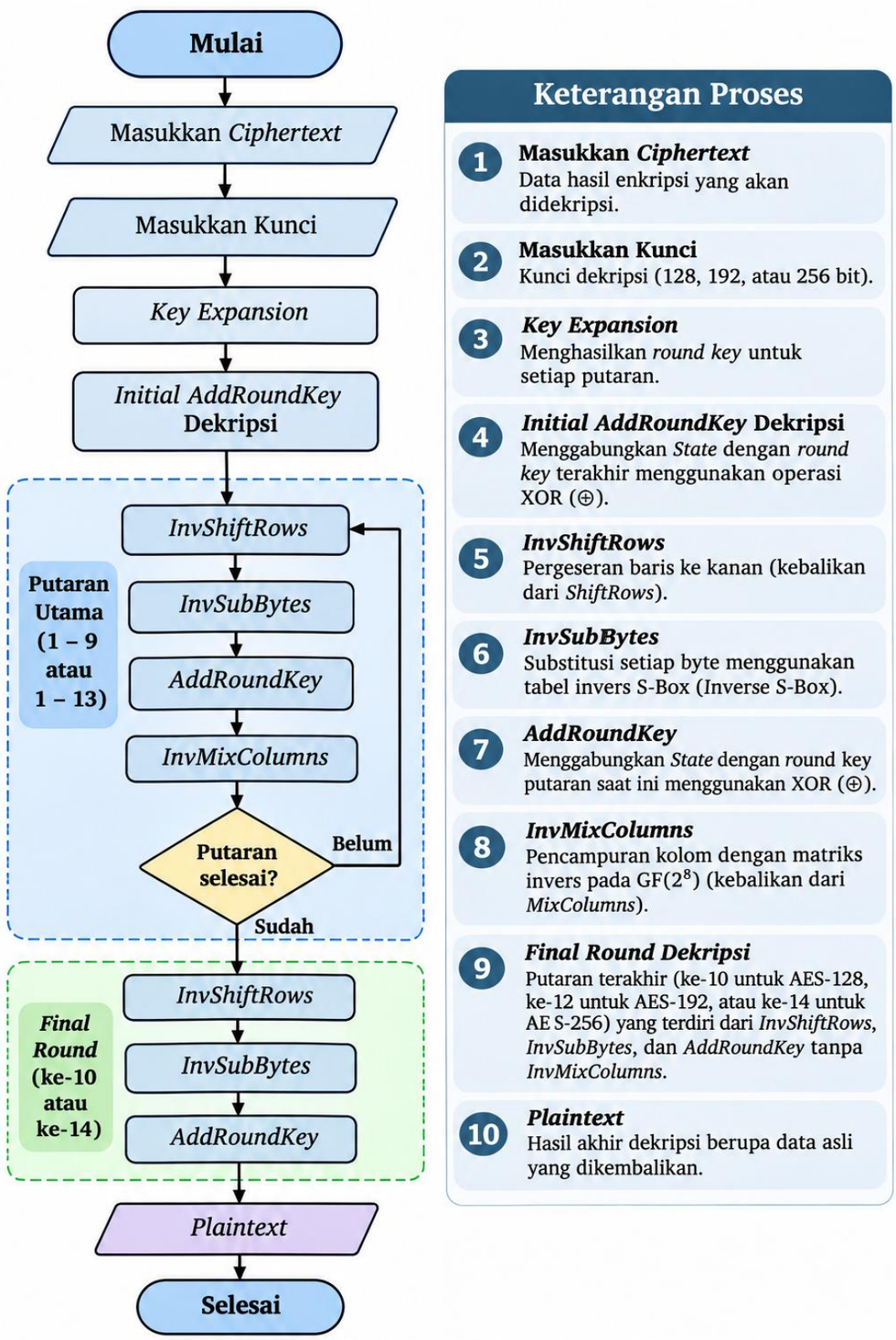
Gambar 3. Diagram Alur Enkripsi AES

Diagram tersebut menunjukkan tahapan utama enkripsi AES. Putaran utama dilakukan berulang sesuai jumlah putaran yang digunakan.

Untuk AES-128 jumlah putarannya adalah 10, AES-192 sebanyak 12, dan AES-256 sebanyak 14.

3.2 Diagram Alur Dekripsi AES

Proses dekripsi dilakukan untuk mengembalikan ciphertext menjadi plaintext. Proses ini menggunakan round key yang sesuai dan transformasi invers.



Gambar 4. Diagram Alur Dekripsi AES

Diagram ini menggambarkan urutan konseptual proses dekripsi AES. Pada implementasi standar, penggunaan round key mengikuti urutan yang sesuai dengan struktur dekripsi AES.

3.3 Analisis Key Expansion

Key Expansion merupakan bagian penting dalam AES karena menghasilkan round key yang digunakan pada setiap putaran.

Proses ini dimulai dari cipher key yang diberikan sebagai masukan. Selanjutnya, kunci dipecah menjadi beberapa word berukuran 32 bit.

Jumlah word awal bergantung pada panjang kunci:

Tabel 5. Analisis Key Expansion

Jenis AES	Jumlah Word Kunci Awal
AES-128	4
AES-192	6
AES-256	8

Setelah itu, algoritma menghasilkan word tambahan melalui kombinasi operasi XOR, RotWord, SubWord, dan Rcon.

Secara umum, hasil Key Expansion digunakan untuk menyediakan round key yang berbeda pada setiap putaran.

Pada AES-128, jumlah keseluruhan round key adalah 11, termasuk round key awal. AES-192 memiliki 13 round key, sedangkan AES-256 memiliki 15 round key.

Dengan demikian, semakin panjang kunci yang digunakan, semakin banyak round key yang dihasilkan oleh proses Key Expansion.

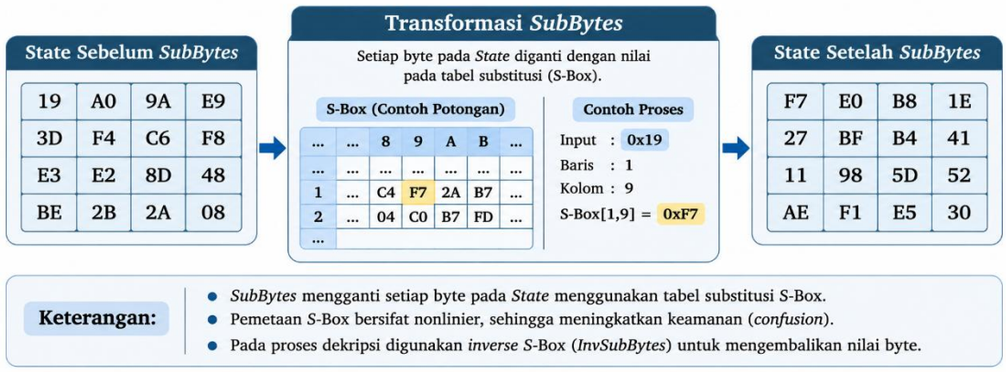
3.4 Analisis SubBytes

SubBytes merupakan transformasi yang mengganti setiap byte pada State menggunakan S-Box.

Proses ini tidak sekadar mengganti karakter berdasarkan tabel biasa, tetapi menggunakan pemetaan matematis yang dirancang agar menghasilkan transformasi nonlinier.

Sifat nonlinier tersebut penting karena dapat mengurangi hubungan sederhana antara nilai input dan output.

Pada proses dekripsi, nilai byte dikembalikan menggunakan inverse S-Box.



Gambar 5. Ilustrasi Transformasi SubBytes

3.5 Analisis ShiftRows

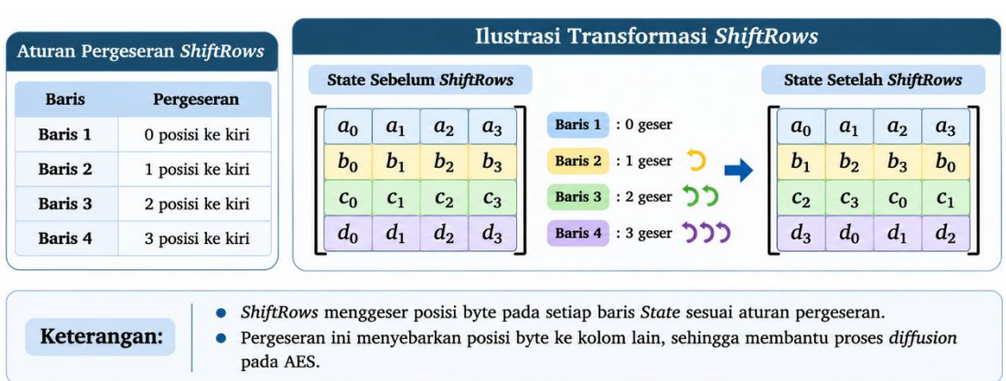
ShiftRows mengubah posisi byte pada setiap baris State. Transformasi ini menyebabkan susunan data berubah sehingga byte tidak tetap berada dalam posisi kolom awal.

Pergeseran baris dilakukan dengan pola berikut:

Tabel 6. Analisis ShiftRows

Baris	Pergeseran
Baris 1	0 posisi ke kiri
Baris 2	1 posisi ke kiri
Baris 3	2 posisi ke kiri
Baris 4	3 posisi ke kiri

Pergeseran tersebut menyebabkan byte dari satu kolom berpindah ke kolom lainnya. Proses ini bekerja bersama MixColumns untuk menyebarkan pengaruh perubahan data.



Gambar 6. Ilustrasi Transformasi ShiftRows

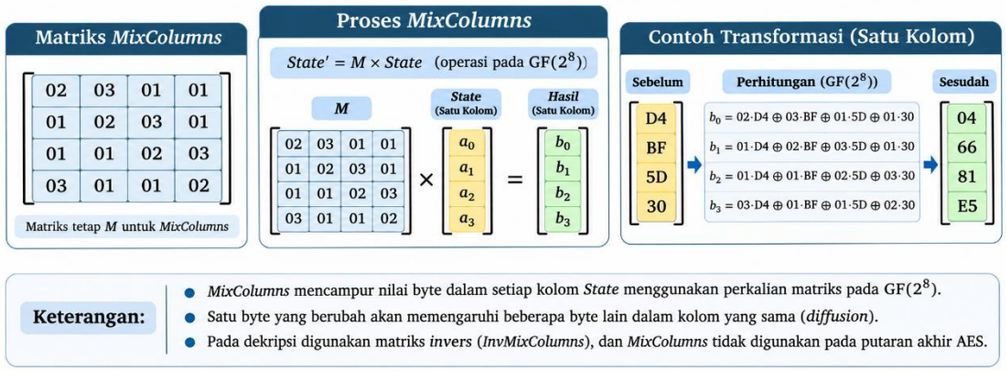
3.6 Analisis MixColumns

MixColumns melakukan pencampuran nilai byte dalam setiap kolom State menggunakan operasi matematika pada (GF(2^8)).

Transformasi ini menyebabkan perubahan pada satu byte dapat memengaruhi beberapa byte lain dalam kolom yang sama.

MixColumns menggunakan perkalian matriks tetap. Pada proses dekripsi, transformasi tersebut dibalik menggunakan InvMixColumns.

MixColumns tidak digunakan pada putaran akhir AES. Hal ini menjadi salah satu perbedaan antara putaran tengah dan putaran terakhir.



Gambar 7. Ilustrasi Transformasi MixColumns

3.7 Analisis AddRoundKey

AddRoundKey menggabungkan State dengan round key menggunakan operasi XOR.

Tahapan ini merupakan satu-satunya transformasi utama AES yang secara langsung menggabungkan State dengan round key.

Karena proses XOR memiliki sifat:

$$A \oplus B = C$$

dan

$$C \oplus B = A$$

maka operasi XOR dapat digunakan kembali untuk mengembalikan nilai sebelumnya selama round key yang digunakan benar.

AddRoundKey dilakukan pada proses awal, setiap putaran utama, dan putaran akhir.

3.8 Analisis Perbedaan AES-128, AES-192, dan AES-256

Perbedaan ketiga variasi AES dapat dianalisis dari panjang kunci, jumlah putaran, dan jumlah round key.

Tabel 7. Analisis Perbedaan AES-128, AES-192, dan AES-256

Aspek	AES-128	AES-192	AES-256
Panjang kunci	128 bit	192 bit	256 bit
Ukuran kunci	16 byte	24 byte	32 byte
Ukuran blok	128 bit	128 bit	128 bit
Jumlah putaran	10	12	14
Jumlah round key	11	13	15

Dari tabel tersebut terlihat bahwa ukuran blok tidak mengalami perubahan. Perbedaan utama terdapat pada panjang kunci dan jumlah putaran.

AES-256 menggunakan kunci yang lebih panjang dibandingkan AES-128 dan AES-192. Hal ini memberikan ruang kemungkinan kunci yang lebih besar, tetapi tidak berarti bahwa seluruh aspek keamanan sistem otomatis menjadi lebih baik. Keamanan tetap dipengaruhi oleh mode operasi, pembangkitan kunci, pengelolaan nonce, dan cara implementasi.

3.9 Analisis Hubungan Enkripsi dan Dekripsi

Enkripsi dan dekripsi AES merupakan dua proses yang saling berhubungan.

Enkripsi menggunakan plaintext dan kunci untuk menghasilkan ciphertext. Dekripsi menggunakan ciphertext dan kunci yang sesuai untuk memperoleh kembali plaintext.

Hubungan keduanya dapat dituliskan sebagai:

$$D_K\big(E_K(P)\big) = P$$

Persamaan tersebut menunjukkan bahwa hasil dekripsi dari ciphertext yang benar harus sama dengan plaintext awal apabila kunci dan proses yang digunakan sesuai.

Prinsip ini menjadi dasar verifikasi dalam project. Program tidak hanya menampilkan ciphertext, tetapi juga melakukan dekripsi dan membandingkan hasilnya dengan teks awal.

BAB IV
METODE DAN STUDI KASUS

4.1 Metode Pengerjaan

Project ini dikerjakan melalui beberapa tahapan, mulai dari studi literatur, analisis algoritma, persiapan lingkungan pemrograman, implementasi, hingga pengujian.

Tahapan pengerjaan adalah sebagai berikut:

1. Mempelajari konsep dasar kriptografi simetris dan AES.
2. Mengidentifikasi perbedaan AES-128, AES-192, dan AES-256.
3. Menganalisis transformasi utama AES.
4. Menentukan bahasa pemrograman yang digunakan.
5. Menyiapkan lingkungan Python dan pustaka kriptografi.
6. Membuat program enkripsi dan dekripsi menggunakan AES-256-GCM.
7. Menjalankan program menggunakan data uji.
8. Membuat unit test untuk menguji proses enkripsi, dekripsi, dan autentikasi.
9. Mendokumentasikan hasil implementasi dan pengujian.
10. Menganalisis hasil serta keterbatasan program.

4.2 Lingkungan Pengembangan

Implementasi project dilakukan menggunakan bahasa pemrograman Python dengan pustaka cryptography.

Lingkungan pengembangan yang digunakan adalah sebagai berikut:

Tabel 8. Lingkungan Pengembangan

Komponen	Keterangan
Bahasa pemrograman	Python
Versi Python	3.13.2
Library kriptografi	cryptography
Versi library	50.0.2
Algoritma	AES-256
Mode operasi	Galois/Counter Mode (GCM)

Editor	[ISI EDITOR YANG DIGUNAKAN]
Sistem operasi	Windows
Pengujian	Python unittest

Pustaka cryptography digunakan karena menyediakan implementasi algoritma kriptografi yang telah tersedia dan dapat digunakan melalui antarmuka pemrograman.

Penggunaan pustaka tepercaya lebih tepat dibandingkan membuat algoritma kriptografi sendiri untuk penggunaan nyata, karena implementasi kriptografi membutuhkan ketelitian dan pengujian keamanan yang tinggi [4].

4.3 Studi Kasus

Studi kasus dalam project ini adalah melakukan pengamanan terhadap sebuah teks menggunakan algoritma AES-256-GCM.

Teks yang digunakan merupakan data uji yang tidak mengandung informasi rahasia. Data tersebut digunakan untuk memperlihatkan proses perubahan plaintext menjadi ciphertext dan proses pengembalian ciphertext menjadi plaintext.

Plaintext yang digunakan adalah:

```
ILHAM FIRMANSYAH - UTS KRIPTOGRAFI
```

Teks tersebut dipilih karena dapat digunakan untuk menguji proses encoding UTF-8, enkripsi, dekripsi, dan verifikasi kesamaan hasil.

4.3.1 Data Plaintext

Plaintext merupakan teks asli yang menjadi masukan proses enkripsi.

Atribut	Nilai
Jenis data	Teks
Plaintext	ILHAM FIRMANSYAH - UTS KRIPTOGRAFI
Encoding	UTF-8

Sebelum diproses oleh algoritma AES, teks diubah menjadi kumpulan byte menggunakan encoding UTF-8.

Representasi heksadesimal dari plaintext tersebut adalah:

```
494c48414d204649524d414e53594148202d20555453204b524950544f4752414649
```

Nilai tersebut merupakan representasi byte dari teks yang digunakan oleh program.

4.3.2 Pembangkitan Kunci

Program menggunakan AES-256 sehingga kunci yang dibutuhkan memiliki panjang 256 bit atau 32 byte.

Kunci dibangkitkan menggunakan fungsi AESGCM.generate_key() yang disediakan oleh pustaka cryptography.

Pembangkitan dilakukan secara acak sehingga kunci yang dihasilkan dapat berbeda pada setiap eksekusi program.

Dalam implementasi ini, nilai kunci tidak ditampilkan secara langsung pada output terminal. Program hanya menampilkan informasi panjang kunci. Pendekatan tersebut digunakan agar laporan tidak memuat nilai kunci yang tidak diperlukan untuk dokumentasi hasil.

4.3.3 Pembangkitan Nonce

Selain kunci, AES-GCM membutuhkan nonce. Pada program ini, nonce memiliki panjang 12 byte atau 96 bit.

Nonce dibangkitkan menggunakan fungsi os.urandom().

Nonce bukan kunci rahasia pengganti key. Nilai tersebut digunakan sebagai parameter proses enkripsi dan harus tersedia kembali ketika proses dekripsi dilakukan.

Untuk penggunaan AES-GCM, nonce harus unik untuk setiap proses enkripsi yang menggunakan kunci yang sama. Penggunaan ulang nonce pada kunci yang sama dapat menyebabkan kelemahan keamanan yang serius [3].

4.3.4 Proses Enkripsi

Setelah plaintext, kunci, dan nonce tersedia, program melakukan proses enkripsi menggunakan AES-GCM.

Secara konseptual, proses tersebut menghasilkan:

1. Ciphertext, yaitu hasil enkripsi plaintext.
2. Authentication tag, yaitu nilai autentikasi untuk memeriksa keabsahan data.

Program kemudian memisahkan hasil ciphertext dan authentication tag agar keduanya dapat ditampilkan secara terpisah.

4.3.5 Proses Dekripsi

Setelah proses enkripsi selesai, program menggabungkan kembali ciphertext dan authentication tag.

Keduanya kemudian diproses menggunakan fungsi dekripsi AES-GCM dengan kunci dan nonce yang sama.

Apabila autentikasi berhasil, program memperoleh kembali plaintext awal.

Jika autentikasi gagal, pustaka akan menghasilkan kesalahan InvalidTag. Hal ini dapat terjadi apabila data atau parameter autentikasi yang digunakan tidak sesuai.

4.4 Representasi Hasil Studi Kasus

Berdasarkan salah satu eksekusi program yang telah dilakukan, diperoleh hasil sebagai berikut.

Tabel 9. Informasi Data Uji

Komponen	Hasil
Plaintext	ILHAM FIRMANSYAH - UTS KRIPTOGRAFI
Encoding	UTF-8
Algoritma	AES-256-GCM
Panjang kunci	256 bit
Panjang nonce	96 bit
Panjang authentication tag	128 bit

Tabel 10. Contoh Hasil Enkripsi

Komponen	Nilai Hasil
Nonce	8DA970944E926565B112D73F
Ciphertext	91A6F611B8EF0176882ED21973D330B7EE7557B4E61CB38D30F9F9C73FB95C8B37A7
Authentication Tag	029F26CC4AA2899F475F4C5B015BF8E3

Catatan: Nilai pada tabel merupakan contoh hasil dari satu kali eksekusi program. Karena kunci dan nonce dibangkitkan secara acak, hasil ciphertext dan authentication tag dapat berbeda pada eksekusi berikutnya.

4.5 Verifikasi Hasil Dekripsi

Setelah proses enkripsi selesai, program melakukan proses dekripsi menggunakan kunci dan nonce yang sama.

Hasil dekripsi pada contoh eksekusi tersebut adalah:

```
ILHAM FIRMANSYAH - UTS KRIPTOGRAFI
```

Hasil tersebut sama dengan plaintext awal yang dimasukkan ke dalam program.

Verifikasi dilakukan melalui perbandingan antara nilai plaintext awal dan hasil dekripsi.

Secara konseptual, kondisi keberhasilan dapat dituliskan sebagai:

$$Plaintext_{awal} = Plaintext_{hasil\ ekripsi}$$

Apabila kedua nilai tersebut sama, maka proses enkripsi dan dekripsi berhasil dilakukan untuk data uji tersebut.

Hasil program menunjukkan status:

```
Status       : SUCCESS
Verification: PLAINTEXT MATCH
```

Berdasarkan hasil tersebut, program berhasil mengembalikan data ke bentuk aslinya setelah melalui proses enkripsi.

BAB V
IMPLEMENTASI DAN PENGUJIAN PROGRAM

5.1 Perancangan Program

Program dibuat menggunakan Python dengan memanfaatkan pustaka cryptography.

Struktur program dipisahkan menjadi beberapa fungsi agar setiap proses memiliki tanggung jawab yang jelas.

Fungsi utama yang digunakan adalah:

1. generate_key(), untuk membangkitkan kunci AES-256.
2. encrypt_message(), untuk melakukan enkripsi plaintext.
3. decrypt_message(), untuk melakukan dekripsi dan verifikasi autentikasi.
4. main(), untuk mengatur alur program dan menampilkan hasil.

Pemisahan fungsi tersebut membuat program lebih mudah dibaca, diuji, dan dikembangkan.

5.2 Struktur Folder Project

Struktur repository yang digunakan adalah sebagai berikut:

```
kriptografi-aes-uts/
│
├── screenshots/
│   ├── aes-gcm-encryption.png
│   └── aes-gcm-tests.png
│
├── src/
│   ├── __init__.py
│   └── main.py
│
├── tests/
│   └── test_aes.py
│
├── .gitignore
├── README.md
└── requirements.txt
```

Keterangan:

- src/main.py berisi implementasi utama AES-256-GCM.
- tests/test_aes.py berisi pengujian otomatis.
- requirements.txt berisi daftar dependensi Python.
- .gitignore digunakan untuk mengabaikan file yang tidak perlu dimasukkan ke repository.
- README.md berisi dokumentasi project dan petunjuk menjalankan program.
- screenshots/ digunakan untuk menyimpan bukti hasil eksekusi dan pengujian.

5.3 Persiapan Lingkungan Python

Sebelum program dijalankan, diperlukan lingkungan Python dan pustaka yang mendukung proses kriptografi.

Tahapan persiapannya adalah sebagai berikut.

1. Memeriksa Python

Python diperiksa menggunakan perintah:

```
python --version
```

Pada lingkungan pengembangan project ini, versi Python yang digunakan adalah 3.13.2.

2. Membuat Virtual Environment

Virtual environment dibuat agar dependensi project terpisah dari instalasi Python secara umum.

```
python -m venv .venv
```

3. Mengaktifkan Virtual Environment

```
.venv\Scripts\Activate.ps1
```

4. Memasang Library

```
python -m pip install cryptography
```

5. Menyimpan Dependensi

```
python -m pip freeze > requirements.txt
```

Setelah tahapan tersebut selesai, lingkungan pengembangan siap digunakan untuk menjalankan program.

5.4 Implementasi Fungsi Pembangkitan Kunci

Fungsi generate_key() digunakan untuk membuat kunci AES dengan panjang 256 bit.

Potongan implementasinya adalah:

```
KEY_SIZE = 256


def generate_key() -> bytes:
    return AESGCM.generate_key(bit_length=KEY_SIZE)
```

Fungsi tersebut menggunakan fasilitas pembangkitan kunci dari kelas AESGCM.

Nilai kunci dikembalikan dalam bentuk bytes sehingga dapat digunakan secara langsung oleh proses enkripsi dan dekripsi.

5.5 Implementasi Fungsi Enkripsi

Fungsi encrypt_message() digunakan untuk mengubah plaintext menjadi ciphertext.

Potongan implementasinya adalah:

```
def encrypt_message(
    plaintext: str,
    key: bytes,
    nonce: bytes
) -> tuple[bytes, bytes]:

    if len(nonce) != NONCE_SIZE:
        raise ValueError("Nonce must be exactly 12 bytes.")

    aesgcm = AESGCM(key)

    encrypted_data = aesgcm.encrypt(
        nonce,
        plaintext.encode("utf-8"),
        None
    )

    ciphertext = encrypted_data[:-TAG_SIZE]
    authentication_tag = encrypted_data[-TAG_SIZE:]

    return ciphertext, authentication_tag
```

Pada fungsi tersebut terdapat beberapa proses.

Pertama, program memeriksa panjang nonce agar sesuai dengan ukuran yang ditentukan.

Kedua, objek AESGCM dibuat menggunakan kunci yang telah dibangkitkan.

Ketiga, plaintext dikonversi menjadi bytes menggunakan UTF-8.

Keempat, fungsi encrypt() melakukan proses enkripsi dan menghasilkan data terenkripsi beserta authentication tag.

Terakhir, hasil enkripsi dipisahkan menjadi ciphertext dan authentication tag untuk memudahkan proses penampilan dan pengujian.

5.6 Implementasi Fungsi Dekripsi

Fungsi decrypt_message() digunakan untuk mengembalikan ciphertext menjadi plaintext.

Potongan implementasinya adalah:

```
def decrypt_message(
    ciphertext: bytes,
    authentication_tag: bytes,
    key: bytes,
    nonce: bytes
) -> str:

    if len(nonce) != NONCE_SIZE:
        raise ValueError("Nonce must be exactly 12 bytes.")

    if len(authentication_tag) != TAG_SIZE:
        raise ValueError("Authentication tag must be 16 bytes.")

    aesgcm = AESGCM(key)

    encrypted_data = ciphertext + authentication_tag

    plaintext = aesgcm.decrypt(
        nonce,
        encrypted_data,
        None
    )

    return plaintext.decode("utf-8")
```

Fungsi tersebut melakukan pemeriksaan panjang nonce dan authentication tag sebelum proses dekripsi dijalankan.

Ciphertext dan authentication tag kemudian digabungkan kembali sesuai format yang diperlukan oleh fungsi dekripsi.

Setelah itu, program melakukan autentikasi dan dekripsi menggunakan kunci serta nonce yang sesuai.

Apabila proses berhasil, hasil dekripsi dikonversi kembali menjadi string menggunakan UTF-8.

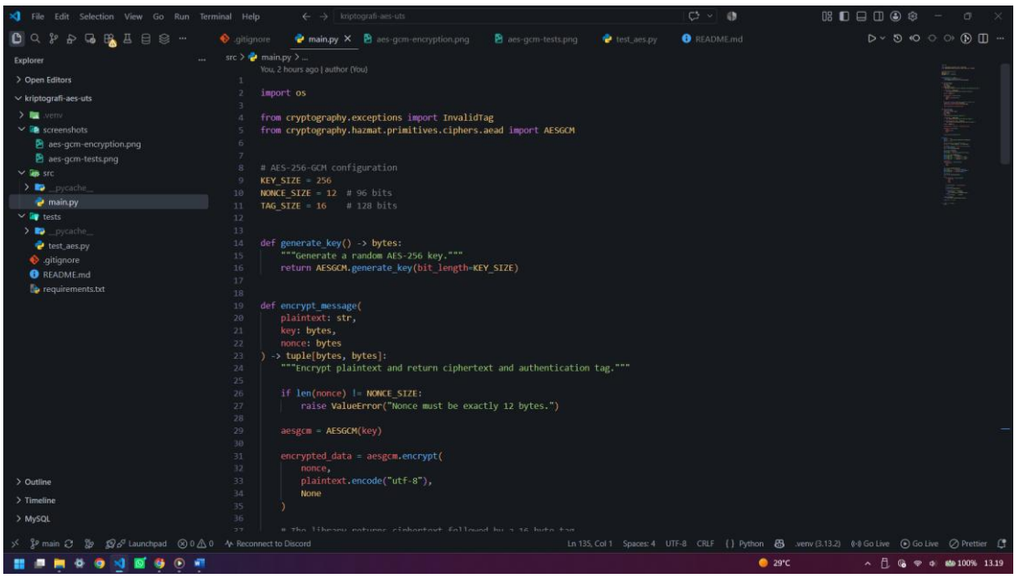
Apabila autentikasi tidak berhasil, pustaka akan menghasilkan kesalahan InvalidTag.

5.7 Implementasi Alur Program Utama

Fungsi main() mengatur keseluruhan proses program, mulai dari menentukan plaintext, membangkitkan kunci dan nonce, melakukan enkripsi, melakukan dekripsi, hingga menampilkan hasil verifikasi.

Alur tersebut dapat dijelaskan sebagai berikut:

1. Program menampilkan judul aplikasi.
2. Program menentukan plaintext yang digunakan sebagai data uji.
3. Program membangkitkan kunci AES-256.
4. Program membangkitkan nonce sepanjang 12 byte.
5. Program menampilkan informasi plaintext dan representasi byte.
6. Program melakukan enkripsi.
7. Program menampilkan nonce, ciphertext, dan authentication tag.
8. Program melakukan dekripsi.
9. Program membandingkan hasil dekripsi dengan plaintext awal.
10. Program menampilkan status keberhasilan atau kegagalan.



Gambar 8. Tampilan Program AES-256-GCM

5.8 Implementasi Unit Testing

Selain menjalankan program utama, dilakukan pengujian otomatis menggunakan modul unittest dari Python.

Pengujian dibuat untuk memastikan bahwa program tidak hanya berhasil melakukan enkripsi dan dekripsi, tetapi juga dapat mendeteksi perubahan terhadap data dan parameter autentikasi.

Pengujian terdiri dari lima skenario.

5.8.1 Pengujian Enkripsi dan Dekripsi

Pengujian pertama bertujuan memastikan bahwa plaintext yang telah dienkripsi dapat dikembalikan menjadi teks yang sama.

Program melakukan enkripsi menggunakan kunci dan nonce, kemudian melakukan dekripsi menggunakan parameter yang sama.

Hasil dekripsi dibandingkan dengan plaintext awal menggunakan assertEqual().

Jika kedua nilai sama, pengujian dinyatakan berhasil.

5.8.2 Pengujian Perubahan Ciphertext

Pengujian kedua bertujuan memastikan bahwa perubahan pada ciphertext dapat dideteksi.

Program mengubah salah satu byte ciphertext menggunakan operasi XOR, kemudian mencoba melakukan dekripsi.

Karena ciphertext telah berubah, autentikasi diharapkan gagal dan menghasilkan InvalidTag.

5.8.3 Pengujian Perubahan Authentication Tag

Pengujian ketiga dilakukan dengan mengubah salah satu byte authentication tag.

Program kemudian mencoba melakukan dekripsi menggunakan tag yang telah dimodifikasi.

Hasil yang diharapkan adalah kegagalan autentikasi karena tag tidak lagi sesuai dengan data yang dihasilkan pada proses enkripsi.

5.8.4 Pengujian Penggunaan Kunci yang Salah

Pengujian keempat dilakukan dengan membuat kunci yang berbeda dari kunci awal.

Kunci tersebut digunakan untuk mencoba melakukan dekripsi terhadap ciphertext yang telah dihasilkan.

Karena kunci tidak sesuai, proses autentikasi diharapkan gagal dan menghasilkan InvalidTag.

5.8.5 Pengujian Penggunaan Nonce yang Salah

Pengujian kelima dilakukan dengan mengubah salah satu byte nonce.

Nonce yang telah diubah kemudian digunakan dalam proses dekripsi.

Hasil yang diharapkan adalah kegagalan autentikasi karena nonce yang digunakan tidak sesuai dengan parameter pada proses enkripsi.

5.9 Hasil Pengujian

Pengujian dilakukan menggunakan perintah:

```
python -m unittest discover -s tests -v
```

Berdasarkan hasil eksekusi, seluruh lima skenario pengujian berhasil dilewati.

Tabel 11. Hasil Pengujian Unit Test

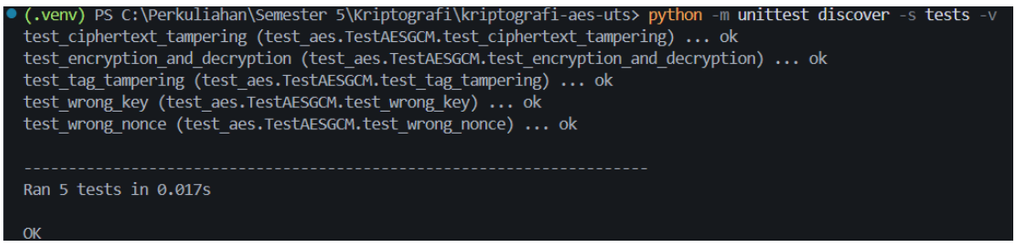
No.	Skenario Pengujian	Hasil yang Diharapkan	Status
1	Enkripsi dan dekripsi	Plaintext kembali sama	Berhasil
2	Perubahan ciphertext	Autentikasi gagal	Berhasil
3	Perubahan authentication tag	Autentikasi gagal	Berhasil
4	Penggunaan kunci yang salah	Autentikasi gagal	Berhasil
5	Penggunaan nonce yang salah	Autentikasi gagal	Berhasil

Hasil terminal menunjukkan:

```
test_ciphertext_tampering ... ok
test_encryption_and_decryption ... ok
test_tag_tampering ... ok
test_wrong_key ... ok
test_wrong_nonce ... ok

Ran 5 tests

OK
```



Gambar 9. Hasil Pengujian Otomatis

5.10 Analisis Hasil Pengujian

Berdasarkan pengujian yang telah dilakukan, terdapat beberapa hal yang dapat dianalisis.

Pertama, proses enkripsi dan dekripsi berhasil dilakukan. Hal ini menunjukkan bahwa program dapat menggunakan AES-256-GCM untuk mengubah plaintext menjadi ciphertext dan mengembalikannya ke bentuk semula.

Kedua, perubahan pada ciphertext menyebabkan proses autentikasi gagal. Hal tersebut menunjukkan bahwa mekanisme GCM dapat mendeteksi perubahan terhadap data yang telah dienkripsi.

Ketiga, perubahan authentication tag juga menyebabkan kegagalan autentikasi. Hal ini memperlihatkan bahwa authentication tag memiliki peran penting dalam memverifikasi keabsahan hasil enkripsi.

Keempat, penggunaan kunci dan nonce yang tidak sesuai menyebabkan proses dekripsi tidak berhasil. Kondisi tersebut menunjukkan bahwa parameter yang digunakan dalam proses dekripsi harus sesuai dengan parameter yang digunakan pada proses enkripsi.

Secara keseluruhan, lima pengujian berhasil dilewati. Namun, keberhasilan pengujian tersebut hanya menunjukkan bahwa skenario yang diuji bekerja sesuai harapan. Hasil tersebut tidak dapat dijadikan bukti bahwa seluruh kemungkinan kelemahan keamanan telah ditemukan atau dihilangkan.

BAB VI
EVALUASI KEAMANAN DAN PEMBAHASAN

6.1 Kerahasiaan Data (Confidentiality)

Kerahasiaan merupakan salah satu tujuan utama penggunaan kriptografi. Data yang dienkripsi diharapkan tidak dapat dibaca secara langsung oleh pihak yang tidak memiliki kunci yang sesuai.

Dalam project ini, plaintext diubah menjadi ciphertext menggunakan AES-256-GCM. Hasil ciphertext ditampilkan dalam bentuk heksadesimal sehingga tidak lagi menyerupai teks asli.

AES-256 menggunakan kunci dengan panjang 256 bit. Kunci tersebut digunakan untuk mengendalikan proses enkripsi dan dekripsi.

Meskipun demikian, kerahasiaan tidak hanya ditentukan oleh panjang kunci. Sistem juga harus memperhatikan keamanan pembangkitan, penyimpanan, dan penggunaan kunci.

6.2 Integritas Data (Integrity)

Integritas merupakan kemampuan untuk memastikan bahwa data tidak mengalami perubahan yang tidak sah.

Dalam AES-GCM, integritas diperiksa melalui authentication tag. Tag tersebut dihasilkan selama proses enkripsi dan diperiksa kembali pada saat dekripsi.

Jika ciphertext atau parameter autentikasi mengalami perubahan, proses verifikasi dapat gagal.

Berdasarkan pengujian project, perubahan ciphertext dan authentication tag menyebabkan proses dekripsi menghasilkan InvalidTag.

Hal ini menunjukkan bahwa mekanisme autentikasi GCM dapat digunakan untuk mendeteksi perubahan terhadap data yang diproses [3].

6.3 Autentikasi Data

Autentikasi dalam konteks AES-GCM berhubungan dengan pemeriksaan keabsahan ciphertext dan data terkait.

Authentication tag digunakan untuk memverifikasi bahwa data yang diterima masih sesuai dengan proses enkripsi.

Apabila tag tidak sesuai, program tidak boleh memperlakukan hasil dekripsi sebagai plaintext yang valid.

Dalam implementasi project, kesalahan autentikasi ditangani menggunakan exception InvalidTag dari pustaka cryptography.

Hal tersebut membantu program membedakan proses dekripsi yang berhasil dengan proses yang gagal akibat ketidaksesuaian autentikasi.

6.4 Risiko Penggunaan ECB

ECB merupakan salah satu mode operasi yang tidak direkomendasikan untuk mengamankan data dengan pola yang berulang.

Dalam ECB, setiap blok plaintext dienkripsi secara terpisah. Apabila terdapat dua blok plaintext yang sama dan menggunakan kunci yang sama, hasil ciphertext untuk kedua blok tersebut juga akan sama.

Kondisi tersebut dapat memperlihatkan pola tertentu dari data asli meskipun isi data tidak dapat dibaca secara langsung.

Oleh karena itu, ECB tidak dipilih dalam project ini. AES-GCM digunakan karena menyediakan mekanisme kerahasiaan sekaligus autentikasi data.

6.5 Pentingnya Penggunaan Nonce yang Unik

Nonce merupakan salah satu parameter penting dalam penggunaan AES-GCM.

Pada project ini, nonce dibuat dengan panjang 12 byte atau 96 bit menggunakan os.urandom().

Nonce perlu dikelola dengan benar. Untuk setiap enkripsi yang menggunakan kunci sama, nonce harus berbeda atau unik.

Penggunaan ulang nonce pada kunci yang sama dapat merusak jaminan keamanan GCM. Oleh karena itu, sistem yang menggunakan AES-GCM dalam skala besar perlu memiliki mekanisme pengelolaan nonce yang dapat mencegah penggunaan ulang [3].

Dalam project ini, nonce dibangkitkan secara acak untuk setiap eksekusi. Pendekatan tersebut sesuai untuk demonstrasi sederhana, tetapi sistem produksi tetap perlu memperhitungkan risiko tabrakan nonce dan jumlah penggunaan per kunci.

6.6 Pengelolaan Kunci

Kunci merupakan komponen utama dalam kriptografi simetris. Pihak yang memiliki kunci yang sesuai dapat melakukan proses dekripsi terhadap ciphertext.

Oleh karena itu, kunci harus dibangkitkan, disimpan, digunakan, dan didistribusikan secara aman.

Beberapa prinsip pengelolaan kunci yang perlu diperhatikan adalah:

1. Menggunakan sumber pembangkitan kunci yang aman.
2. Tidak menyimpan kunci secara terbuka di repository publik.
3. Tidak membagikan kunci kepada pihak yang tidak berwenang.
4. Menghindari penggunaan kunci yang sama secara sembarangan untuk berbagai kebutuhan.
5. Menyediakan mekanisme penyimpanan dan pengelolaan kunci yang sesuai.
6. Mempertimbangkan pergantian kunci sesuai kebijakan keamanan sistem.

Dalam project ini, kunci dibangkitkan menggunakan fasilitas yang disediakan oleh pustaka cryptography. Program tidak menampilkan nilai kunci secara langsung pada output terminal.

Pendekatan tersebut sesuai untuk demonstrasi, tetapi belum mencakup sistem penyimpanan kunci jangka panjang atau mekanisme pertukaran kunci antarpengguna.

6.7 Kelebihan Implementasi AES-256-GCM

Berdasarkan implementasi yang dilakukan, terdapat beberapa kelebihan dari pendekatan yang digunakan.

1. Menggunakan Algoritma Standar

AES merupakan algoritma yang telah distandardisasi oleh NIST dan banyak digunakan dalam berbagai sistem keamanan informasi.

2. Menggunakan Kunci 256 Bit

Implementasi menggunakan AES-256 sehingga panjang kunci yang digunakan adalah 256 bit.

3. Mendukung Kerahasiaan dan Autentikasi

Mode GCM tidak hanya menghasilkan ciphertext, tetapi juga authentication tag untuk memeriksa keabsahan data.

4. Menggunakan Pustaka Kriptografi

Implementasi menggunakan pustaka cryptography sehingga tidak perlu membuat algoritma AES sendiri.

5. Memiliki Pengujian Otomatis

Program dilengkapi lima unit test yang menguji proses enkripsi, dekripsi, dan penanganan perubahan data.

6.8 Keterbatasan Implementasi

Meskipun program berhasil menjalankan skenario pengujian, masih terdapat beberapa keterbatasan yang perlu diperhatikan.

1. Program hanya menggunakan satu contoh plaintext untuk demonstrasi utama.
2. Program belum menyediakan antarmuka grafis.
3. Program belum mengimplementasikan penyimpanan kunci secara persisten.
4. Program belum menyediakan mekanisme pertukaran kunci antara pengirim dan penerima.
5. Pengujian hanya mencakup lima skenario dan belum mencakup seluruh kemungkinan kesalahan.
6. Program belum melakukan pengukuran performa untuk data berukuran besar.
7. Program belum mengimplementasikan pengelolaan nonce untuk penggunaan jangka panjang atau skala produksi.
8. Program belum menguji interoperabilitas dengan aplikasi lain.

Dengan demikian, program ini lebih tepat digunakan sebagai demonstrasi akademis untuk memahami penerapan AES-256-GCM, bukan sebagai sistem penyimpanan atau komunikasi data produksi.

6.9 Hubungan Analisis AES dan Implementasi AES-GCM

Analisis teori AES dan implementasi AES-GCM dalam project ini memiliki hubungan yang saling melengkapi.

Analisis teori menjelaskan cara kerja inti algoritma AES, mulai dari representasi State, Key Expansion, AddRoundKey, SubBytes, ShiftRows, hingga MixColumns.

Sementara itu, implementasi Python menunjukkan bagaimana algoritma AES digunakan melalui sebuah mode operasi.

Pada implementasi, program tidak membuat sendiri transformasi internal AES. Seluruh proses inti kriptografi ditangani oleh pustaka cryptography.

AES-256 merupakan algoritma block cipher yang digunakan sebagai dasar proses enkripsi, sedangkan GCM merupakan mode operasi yang menambahkan mekanisme autentikasi.

Dengan demikian, analisis transformasi AES memberikan pemahaman mengenai algoritma dasarnya, sedangkan implementasi AES-256-GCM menunjukkan penerapan algoritma tersebut dalam program yang dapat dijalankan.

BAB VII
KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan pembahasan teori, analisis algoritma, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut.

1. Advanced Encryption Standard (AES) merupakan algoritma kriptografi kunci simetris berbasis blok yang menggunakan ukuran blok tetap sebesar 128 bit. AES memiliki tiga variasi panjang kunci, yaitu AES-128, AES-192, dan AES-256 dengan jumlah putaran masing-masing 10, 12, dan 14 putaran.
2. Proses enkripsi AES terdiri dari beberapa transformasi utama, yaitu Key Expansion, AddRoundKey, SubBytes, ShiftRows, dan MixColumns. Setiap transformasi memiliki fungsi yang berbeda dalam mengubah State hingga menghasilkan ciphertext. Proses dekripsi menggunakan transformasi invers untuk mengembalikan ciphertext menjadi plaintext.
3. Implementasi project menggunakan bahasa pemrograman Python dengan pustaka cryptography dan algoritma AES-256-GCM. Program dapat melakukan pembangkitan kunci, pembangkitan nonce, enkripsi, dekripsi, dan menampilkan ciphertext serta authentication tag.
4. Hasil pengujian menunjukkan bahwa proses dekripsi berhasil mengembalikan plaintext yang sama dengan data awal. Hal tersebut membuktikan bahwa proses enkripsi dan dekripsi berhasil dilakukan pada contoh data uji yang digunakan.
5. Lima unit test berhasil dilewati, yaitu pengujian enkripsi dan dekripsi, perubahan ciphertext, perubahan authentication tag, penggunaan kunci yang salah, serta penggunaan nonce yang salah.
6. AES-GCM memberikan kemampuan untuk menjaga kerahasiaan dan mendeteksi perubahan data melalui authentication tag. Namun, penggunaan mode ini tetap memerlukan pengelolaan nonce yang benar, terutama agar tidak terjadi penggunaan ulang nonce pada kunci yang sama.
7. Penggunaan pustaka kriptografi tepercaya lebih tepat dibandingkan membuat algoritma kriptografi sendiri untuk penggunaan nyata. Hal ini karena implementasi kriptografi memerlukan ketelitian, validasi, dan pertimbangan keamanan yang memadai.

Secara keseluruhan, project ini memberikan pemahaman mengenai konsep dan struktur algoritma AES serta penerapannya melalui AES-256-GCM menggunakan Python.

Hasil implementasi menunjukkan bahwa proses enkripsi dan dekripsi dapat berjalan sesuai tujuan pada skenario pengujian yang telah dilakukan.

7.2 Saran

Berdasarkan hasil dan keterbatasan project, terdapat beberapa saran untuk pengembangan selanjutnya.

1. Program dapat dikembangkan dengan menambahkan antarmuka grafis agar lebih mudah digunakan.
2. Implementasi dapat dikembangkan untuk menerima input plaintext dari pengguna secara langsung.
3. Sistem dapat dilengkapi mekanisme penyimpanan dan pengelolaan kunci yang lebih aman.
4. Pengujian dapat diperluas dengan menggunakan variasi panjang plaintext dan berbagai skenario input.
5. Program dapat dikembangkan untuk mengelola proses enkripsi dan dekripsi file.
6. Pengelolaan nonce perlu diperhatikan secara lebih mendalam apabila implementasi digunakan dalam sistem yang melakukan banyak proses enkripsi.
7. Analisis dapat dikembangkan dengan membandingkan performa beberapa ukuran kunci AES dan mode operasi yang berbeda.

DAFTAR PUSTAKA

[1] A. U. Rahman, S. U. Miah, and S. Azad, “Advanced Encryption Standard (AES),” *Pract. Cryptogr. Algorithms Implementations Using C++*, pp. 91–126, 2014, doi: 10.1201/b17707.

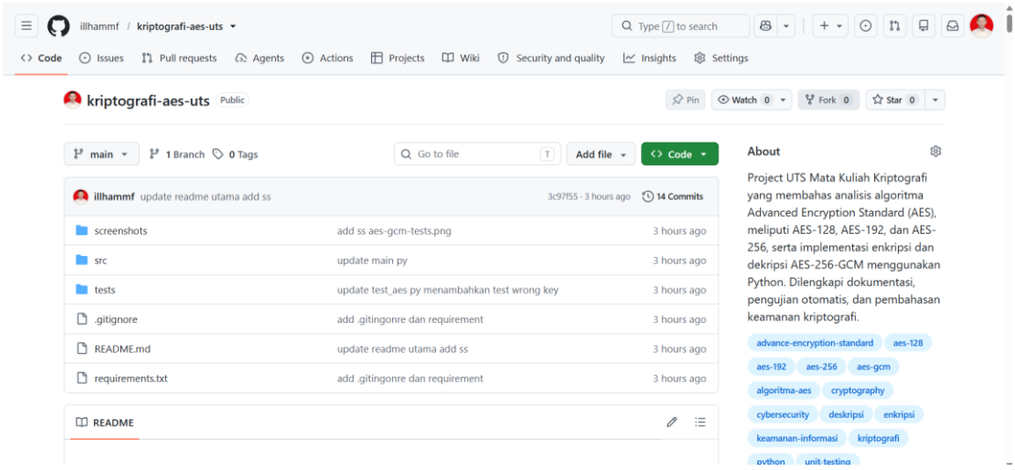
[2] M. J. Dworkin, “Galois/Counter Mode (GCM) and GMAC,” *NIST Spec. Publ. 800-38D*, 2007, [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[3] E. Barker, “Recommendation for key management: Part 1 – General General,” *NIST Spec. Publ. 800-57*, pp. 1–142, 2020, [Online]. Available: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf%0Ahttps://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf

[4] I. Contributors, “Cryptography Documentation,” *Individ. Contrib.*, 2016, [Online]. Available: https://app.readthedocs.org/projects/cryptography/downloads/pdf/stable/

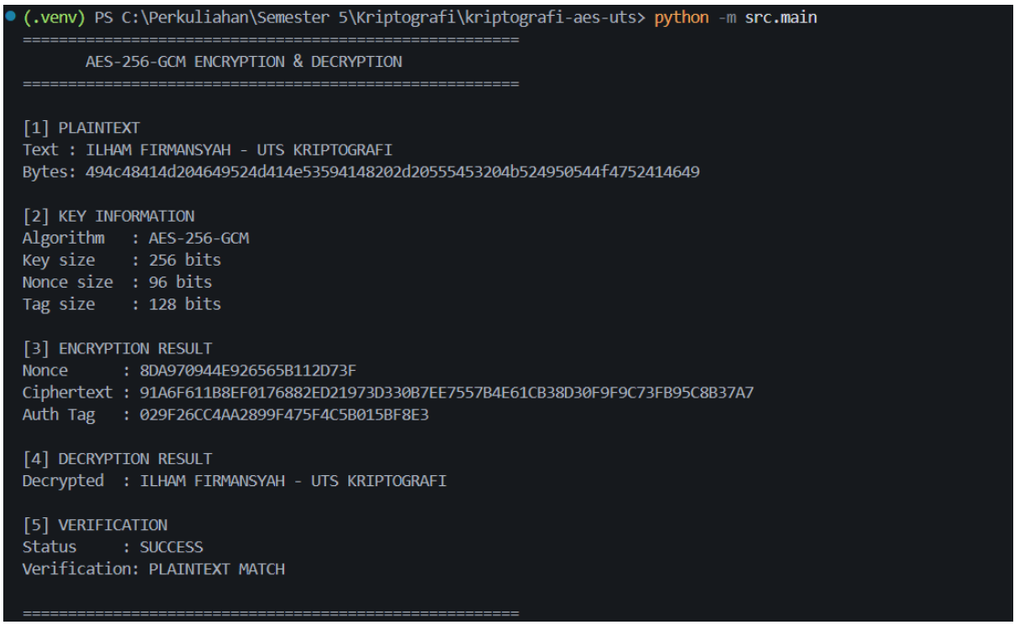
LAMPIRAN

Lampiran A. Dokumentasi Struktur Repository



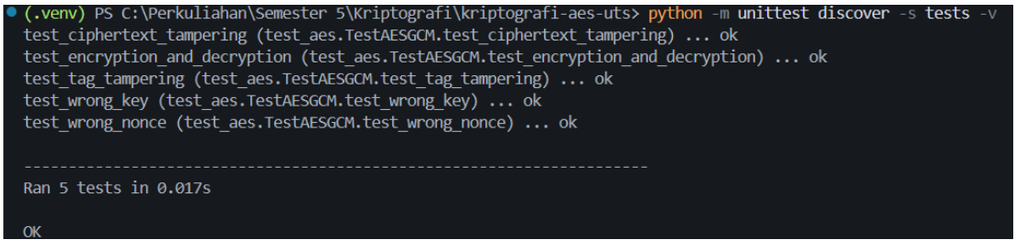
Dokumentasi ini menunjukkan struktur direktori project yang terdiri dari folder source code, unit test, screenshot, dan file konfigurasi.

Lampiran B. Dokumentasi Proses Enkripsi dan Dekripsi



Screenshot ini memperlihatkan proses pembangkitan kunci, nonce, enkripsi plaintext, hasil ciphertext, authentication tag, dan proses dekripsi.

Lampiran C. Dokumentasi Unit Testing



Screenshot ini memperlihatkan hasil lima pengujian otomatis yang telah dijalankan.

Lampiran D. Source Code Program

Source code program dapat dilihat melalui repository GitHub berikut:

Repository:

https://github.com/illhammf/kriptografi-aes-uts

File utama implementasi:

- src/main.py
- tests/test_aes.py
- requirements.txt

Lampiran E. Petunjuk Menjalankan Program

Untuk menjalankan program, pengguna dapat mengikuti langkah berikut:

1. Clone Repository

```
git clone https://github.com/illhammf/kriptografi-aes-uts.git
```

2. Masuk ke Folder Project

```
cd kriptografi-aes-uts
```

3. Membuat Virtual Environment

```
python -m venv .venv
```

4. Mengaktifkan Virtual Environment

Pada Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

5. Memasang Dependensi

```
python -m pip install -r requirements.txt
```

6. Menjalankan Program

```
python -m src.main
```

7. Menjalankan Pengujian

```
python -m unittest discover -s tests -v
```

Apabila seluruh tahapan berhasil dilakukan, program akan menampilkan hasil enkripsi dan dekripsi, sedangkan perintah pengujian akan menampilkan status setiap unit test.